JetBot Nano AG

Spurenverfolgung

1 Einführung

In diesem Abschnitt lernt der JetBot, einer Linie allein anhand seines Kamerabilds zu folgen. Dazu werden zunächst Bilder gesammelt, in denen jeweils der Punkt markiert wird, auf den der Roboter zusteuern soll. Anschließend wird damit ein neuronales Netz trainiert, das der Roboter schließlich im Fahrbetrieb einsetzt, um selbstständig die Strecke abzufahren. Dieses Dokument dient als Begleitmaterial und sollte parallel zu den Notebooks geöffnet werden.

2 Daten sammeln

Öffnet in JupyterLab den Ordner
`Notebooks/road_following/` und darin das Notebook `data_collection.ipynb`.

Führt die Zellen des Notebooks der Reihe nach aus, bis zwei Kamerabilder nebeneinander angezeigt werden. Das linke Bild zeigt das Livebild der Kamera, das rechte den zuletzt gespeicherten Schnappschuss.

Um ein Bild aufzunehmen, klickt ihr im **linken Bild** auf die Stelle, zu der der JetBot fahren soll. Das Bild wird dabei automatisch gespeichert und rechts mit einem grünen Kreis an der angeklickten Stelle angezeigt. Das Feld `count` darunter zählt, wie viele Bilder ihr bereits gesammelt habt.

Wohin klicken?

Im Labor ist auf dem schwarzen Boden mit weißem Tape eine Straße aufgeklebt: zwei Randlinien und eine Mittellinie. Der JetBot soll wie ein Auto in der **rechten Fahrspur** fahren.
Klickt deshalb immer auf die **Mitte der rechten Fahrspur**, also zwischen die Mittellinie und den rechten Rand, und zwar etwas vor dem Roboter. Klickt nicht auf die Linien selbst und nicht direkt unter den Roboter. Wichtig ist, dass alle immer nach derselben Regel klicken, sonst erhält das neuronale Netz widersprüchliche Daten.

Damit das neuronale Netz zuverlässig lernt, braucht es viele und abwechslungsreiche Bilder. Sammelt deshalb **mindestens 300 Bilder** und achtet dabei auf Folgendes:

- Nehmt Bilder an **allen Stellen** der Strecke auf. In den **Kurven** braucht ihr besonders viele, da der Zielpunkt dort deutlich zur Seite wandert.
- Fahrt die Strecke in **der Richtung** ab, in der der JetBot später auch fahren soll. Soll er in beiden Richtungen fahren können, braucht ihr Bilder aus beiden Richtungen.
- Stellt den JetBot nicht nur genau in die Mitte der Spur, sondern auch **näher an die Mittellinie**, **näher an den Rand** oder **schräg**, und klickt trotzdem in die Mitte der

Spur. So lernt er, sich selbst zurück in die Spur zu korrigieren.

- Verändert zwischen den Bildern die Position des Roboters ein wenig, damit keine fast identischen Bilder entstehen.
- Glänzt das Tape im Licht der Deckenlampen, nehmt auch Bilder mit diesen Spiegelungen auf, damit der JetBot sie später nicht mit einer Linie verwechselt.

Mit dem Gamepad fahren

Damit ihr den JetBot beim Sammeln der Bilder nicht ständig hochheben und neu hinstellen müsst, könnt ihr ihn mit dem Gamepad über die Strecke fahren. Fügt dazu im Notebook `data_collection.ipynb` nach der Zelle mit der Kamera zwei neue Zellen ein.

In der ersten Zelle wird das Gamepad verbunden. Führt sie aus und drückt anschließend eine beliebige Taste am Gamepad, bis die Tasten und Achsen angezeigt werden.

```
import ipywidgets.widgets as widgets
from jetbot import Robot

controller = widgets.Controller(index=0)
display(controller)
```

Erst danach führt ihr die zweite Zelle aus. Sie verknüpft das Gamepad mit den Motoren: Mit dem linken Stick gebt ihr Gas, mit dem rechten Stick lenkt ihr.

```
robot = Robot()

SPEED = 0.2
LENKUNG = 0.3

def begrenzen(wert):
    return max(-1.0, min(1.0, wert))

def fahren(change=None):
    gas = -controller.axes[1].value # linker Stick hoch/runter
    lenken = controller.axes[2].value # rechter Stick links/rechts
    robot.left_motor.value = begrenzen(gas * SPEED + lenken * LENKUNG)
    robot.right_motor.value = begrenzen(gas * SPEED - lenken * LENKUNG)

controller.axes[1].observe(fahren, names='value')
controller.axes[2].observe(fahren, names='value')
```

Hinweis

Führt die zweite Zelle nur **einmal** aus. Wollt ihr die Geschwindigkeit ändern, setzt in einer neuen Zelle einfach einen neuen Wert, z. B. `SPEED = 0.3`. Das Gamepad muss im analogen Modus sein: HOME drücken, bis zwei Lämpchen leuchten.

3 Modell trainieren

Nachdem die Bilder gesammelt wurden, geht es nun an das Einlernen der KI. Dabei lernt das neuronale Netz anhand eurer Bilder, für jedes Kamerabild selbst den Zielpunkt vorherzusagen, auf den der JetBot zusteuern soll.

Bevor ihr mit dem Training beginnt, solltet ihr im Notebook `data_collection.ipynb` die Kamera mit `camera.stop()` freigeben und anschließend über *Running* → *Shut Down All* alle laufenden Kernel beenden. Das Training benötigt den gesamten Arbeitsspeicher des Jetson Nano und bricht sonst unter Umständen ab.

Öffnet anschließend im selben Ordner

`Notebooks/road_following/` das Notebook `train_model.ipynb`

und führt die Zellen der Reihe nach aus. Da ihr eure eigenen Bilder verwendet, die bereits im Ordner `dataset_xy` liegen, könnt ihr die Zelle zum Entpacken der Datei `road_following.zip` überspringen. Diese wird nur benötigt, wenn ihr fertige Daten von außerhalb verwendet.

Das Training läuft über mehrere Durchgänge, sogenannte Epochen. Nach jeder Epoche wird angezeigt, wie groß der Fehler des Modells noch ist. Je kleiner dieser Wert wird, desto genauer trifft das Netz den Zielpunkt. Das Training kann je nach Anzahl der Bilder einige Minuten dauern. Das beste Modell wird automatisch in der Datei `best_steering_model_xy.pth` gespeichert, die ihr im nächsten Schritt verwendet.

4 Live-Demo

Nachdem die KI trainiert wurde, öffnet ihr im Ordner

`Notebooks/road_following/` das Notebook `live_demo.ipynb`.

Darin wird euer trainiertes Modell `best_steering_model_xy.pth` geladen, das nach dem Training bereits im selben Ordner liegt. Führt die Zellen der Reihe nach aus. Erst die Zelle mit `camera.observe(execute, names='value')` setzt den JetBot in Bewegung. Stellt ihn vorher in die rechte Fahrspur und haltet ihn im Zweifel fest. Mit der Zelle darunter könnt ihr ihn jederzeit wieder anhalten.

Die Regler

Im Notebook werden zwei Gruppen von Reglern angezeigt. Mit der ersten Gruppe stellt ihr das Fahrverhalten ein:

- **speed gain**: Grundgeschwindigkeit des JetBot. Steht der Regler auf 0, fährt er nicht los.
- **steering gain**: Wie stark der JetBot lenkt, wenn der Zielpunkt seitlich liegt. Ist der Wert zu klein, fährt er in Kurven geradeaus aus der Spur. Ist er zu groß, schlängelt er sich hin und her.
- **steering kd**: Dämpfung der Lenkung. Dieser Wert reagiert darauf, wie schnell sich die Richtung ändert, und kann ein Hin-und-her-Pendeln abschwächen. Ist er zu groß, lenkt der JetBot ruckartig.
- **steering bias**: Ausgleich für einen Roboter, der immer zur selben Seite zieht, zum Beispiel weil ein Motor etwas stärker ist als der andere.

Die zweite Gruppe zeigt nur an, was der JetBot gerade „denkt“, und lässt sich nicht verstellen: **x** und **y** sind der vom Modell erkannte Zielpunkt, **steering** ist der daraus berechnete Lenkwert und **speed** die aktuelle Geschwindigkeit. Liegt der Zielpunkt genau vor dem Roboter, steht **x** nahe bei 0.

Aus diesen Werten berechnet das Notebook für jedes Kamerabild die Lenkung und daraus die Geschwindigkeit der beiden Räder:

$$\text{steering} = \text{Winkel} \cdot \text{steering gain} + \text{Winkeländerung} \cdot \text{steering kd} + \text{steering bias}$$

$$\text{linkes Rad} = \text{speed} + \text{steering}, \qquad \text{rechtes Rad} = \text{speed} - \text{steering}$$

Regler einstellen

Beginnt mit diesen Startwerten und verändert immer nur **einen** Regler in kleinen Schritten:

Regler	**Startwert**
speed gain	0.15 bis 0.20
steering gain	0.15
steering kd	0.00
steering bias	0.00

1. **steering gain** anpassen: Verlässt der JetBot in Kurven die Spur, erhöht den Wert in Schritten von 0.02. Schlängelt er sich, verringert ihn.
2. **steering kd** hinzunehmen, wenn er trotz passender Lenkung noch pendelt. Beginnt mit 0.01 und erhöht vorsichtig.
3. **steering bias** einstellen, wenn er auf gerader Strecke immer zur selben Seite zieht. Zieht er nach rechts, schiebt den Regler ins Negative, zieht er nach links, ins Positive.
4. **speed gain** erst zum Schluss langsam erhöhen. Bei höherem Tempo müsst ihr die Lenkung meist etwas nachstellen.

Fährt er trotzdem nicht sauber?

Beobachtet den Regler **x**. Springt er stark hin und her, obwohl der JetBot ruhig in der Spur steht, liegt das Problem nicht an den Reglern, sondern an den Trainingsdaten. Sammelt dann an den Problemstellen weitere Bilder und trainiert das Modell erneut.